72.75: Aprendizaje Automático (Machine Learning)

Horario: Miércoles 16-19h- virtual sincrónico

Profesorado de la asignatura:
Francisco Sammartino, Ing. | frperez@itba.edu.ar
Alexa Sessarego Aurich
Carlos Bibián Nogueras, PhD | cbibian@itba.edu.ar

Plan de la sesión de hoy

- Algoritmos avanzados
- MLOps
- Deploy
- Mantenimiento y Monitoreo
- TP3

… en la sesión anterior

- Validación cruzada avanzada
- ML diagnostics
- Data-Centered Design
- Análisis de Errores

TP3 y ML competition
Reglas y evaluación

Entregables

Cada equipo deberá entregar:

Función predictor, que se utilizará para evaluar el modelo en la competición.

Notebook y presentación (PPT), siguiendo el formato habitual de los trabajos prácticos.

Fechas de submission

Cada equipo tendrá tres oportunidades de submission, con actualización del leaderboard en ambas fechas: 04/11, 11/11 y 18/11 (antes de las 16:00 h)

Competition Score: La clasificación final se calculará como:

Competition Score = 90% × Score (submission del 18/11) + 10% × Nota de presentación

Premios

Los equipos que alcancen las siguientes posiciones quedarán eximidos del examen final de diciembre:

TOP 1 – Model Score: Equipo con el mejor rendimiento predictivo.

TOP 4 – Competition Score: Los cuatro equipos mejor clasificados según el puntaje combinado.

Condición: Los equipos premiados deberán realizar una breve presentación final el 25/11, último día de clase (técnica + no técnica).

Gradient Boosting

Carlos Bibián Nogueras, PhD
cbibian@itba.edu.ar |

Instituto Tecnológico de Buenos Aires

Gradient Boosting
Idea principal

Los árboles no se entrenan de forma independiente sino que lo hacen de forma secuencial

Cada uno, corrige errores del anterior: El modelo aprende iterativamente sobre los residuos

Por eso se llama Gradient Boosting, ya que reduce progresivamente el error

Gradient Boosting
Funcionamiento

1- Entrena el primer árbol, que genera una predicción inicial sobre los datos

2- Entrena un segundo árbol sobre los errores del anterior

3- Se entrena un tercer árbol sobre los errores del ensemble 1-2

4- Repite este proceso sucesivamente

- De esta forma el ensemble se vuelve cada vez más preciso

Gradient Boosting
Mejora con respecto a RF

Los árboles ahora interactúan en lugar de ser independientes

El modelo se enfoca explícitamente en ejemplos difíciles

Logra mejores precisiones con menos árboles (y por tanto es más rápido y eficiente)

Especialmente potentes en datasets tabulares (igual que RF), ya que se maneja muy bien con datasets de variables heterogéneas

- ya que cada variable se mira por separado en un DT

Reduce el sesgo más que RF. **En RF al añadir árboles reducimos la varianza, pero no tanto el sesgo**.

Gradient Boosting, al enfocarse sucesivamente en errores, ataca el sesgo.

Ejemplos de Gradient Boosting
XGBoost

Mejora el gradient boosting clásico agregando regularización explícita para evitar overfitting.

- Penaliza árboles demasiado complejos (muchas hojas)
- Minimiza no solo el error de predicción, sino también la complejidad del modelo

Cada nuevo árbol **corrige los errores del ensemble previo** minimizando una función de pérdida diferenciable - lo que facilita la optimización

Introduce optimizaciones de cómputo como: paralelización, manejo eficiente de memoria y tratamiento automático de missing values.

Es un **estándar en la industria en ML tabular** porque logra **alta precisión** con relativamente **poco tuning** y **datasets medianos**.

Ejemplos de Gradient Boosting
LightGBM



Desarrollado por **Microsoft** para entrenar boosting sobre **datasets extremadamente grandes**.

Prioriza la velocidad de cómputo y escalabilidad:

En lugar de evaluar todos los posibles splits, agrupa valores en histogramas, acelerando mucho el entrenamiento.

Usa **crecimiento leaf-wise**: expande primero las hojas con mayor reducción de error en vez de crecer el árbol de forma balanceada.

- no entrena cada nivel de profundidad por separado, sino que profundiza en las ramas más prometedoras primero

Esto suele mejorar precisión y velocidad, aunque puede aumentar riesgo de overfitting si no se regulariza bien. (**cuidado al usarlo en datasets pequeños!**)

Ejemplos de Gradient Boosting
CatBoost



Desarrollado por **Yandex** para manejar variables categóricas de forma más robusta.

Fue diseñado para evitar problemas al codificar categorías

CatBoost utiliza **ordered boosting y codificaciones estadísticas** controladas para evitar usar información futura durante entrenamiento.

Esto permite trabajar directamente con muchas variables categóricas reduciendo preprocessing manual y mejorando estabilidad.

Por eso CatBoost suele funcionar muy bien en datasets empresariales/tabulares reales.

Gradient Boosting en Scikit-Learn

Scikit-learn sí tiene boosting clásico:

- GradientBoostingClassifier
- GradientBoostingRegressor
- HistGradientBoosting

Este último (HistGradientBoosting) está bastante inspirado en ideas modernas tipo LightGBM:

- histogram bins,
- entrenamiento más rápido,
- mejor escalabilidad.

Pero XGBoost, LightGBM y CatBoost no estana incluidos y deben ser instalados aparte si se quieren usar

Métodos avanzados basados en vecinos

Carlos Bibián Nogueras, PhD
cbibian@itba.edu.ar |

Instituto Tecnológico de Buenos Aires

k-vecinos más cercanos (knn)

k-nearest neighbours (knn)

- Algoritmo más característicos de esta familia

- Dada una nueva instancia, **identifica los k vecinos más cercanos en el conjunto de entrenamiento, y le asigna la clase más frecuente de esos vecinos**.

- K es el numero de vecinos a considerar (hiperparámetro!)- cuantos vecinos mirar

- Simple pero muy competitivo y capaz de captar estructuras complejas.

Métodos basados en vecinos
Limitaciones

Coste computacional elevado (y en el momento de la predicción!)

- Debe calcular las distancias de cada punto a estimar/clasificar con TODOS los puntos del dataset

La comparación de distancias no es tan efectiva en espacios de alta dimensionalidad (todas las dimensiones tienen la misma importancia, por lo que la información se diluye)

- Este factor afecta mucho menos a los métodos basados en árboles, ya que aqui cada feature se evalúa por separado en cada nodo

Vecinos más cercanos approximados (ANN)

Approximate Nearest Neighbors (ANN)

LSH: Locality-Sensitive Hashing *(hashing: picar/dividir en trozos)*

Busca una división (hashing) del espacio donde la cercanía (locality) se mantenga

Pasos:

1. Generamos k **hiperplanos aleatorios** y vemos cada punto **a que lado de cada hiperplano está**, obteniendo un vector hash para cada punto
   -> **h(x) = (1,0,0,0,1,0)**

2. Cuando tenemos que clasificar un punto, calculamos a qué **bucket** pertenece: **puntos que compartan el mismo h(x)**.

3. Podemos crear varios **(L) proyecciones a h(x)** con distintos gruposde planos aleatorios, en ese caso, calcularemos los **buckets a los que pertenece el punto en cada espacio hash distinto** y "marcaremos" los puntos están en el **mismo bucket al menos una vez.**

4. **Sobre esos puntos**, calculamos la distancia (**en el espacio original de features**, no en el hash!) y aplicamos knn

Hierarchical Navigable Small World (HNSW)

Es uno de los métodos ANN más usados actualmente.

Construye un grafo donde puntos similares quedan conectados.

Utiliza distintos niveles de grafos, de forma que los niveles más altos permiten saltos largos y los más bajos refinan la búsqueda localmente

La búsqueda navega el grafo hacia regiones cada vez más cercanas.

Logra muy buena precisión con latencia extremadamente baja.

Facebook AI Similarity Search (FAISS)



Desarrollado por **Meta** para búsqueda eficiente de similitud.

Permite manejar millones o billones de embeddings con optimizaciones de cómputo sobre ANN

Embeddings: Objetos complejos se representan como vectores densos.

En ese espacio, similaridad geométrica implica similaridad semántica: Dos canciones cercanas en el espacio vectorial “suenan parecidas”.

ANN hace posible buscar rápidamente en esos espacios.

Hoy es infraestructura **estándar en sistemas de retrieval** - recuperar información relevante (buscadores, sistemas de recomendación de productos similares…)

Ejemplo Spotify - ANNOY
Approximate Nearest Neighbors Oh Yeah

Se crea para un contexto muy concreto: recomendar canciones similares de una base de datos de millones de canciones a millones de usuarios y en tiempo real

Implicaciones:

- Miles de búsquedas por segundo
- Construcción razonablemente rápida (se actualiza frecuentemente)
- Indexar nuevos ítems sin rehacer todo (cada nuevo usuario reentrenar todo?!?)

Combinación de modelos

Carlos Bibián Nogueras, PhD
cbibian@itba.edu.ar |

Instituto Tecnológico de Buenos Aires

Combinación de modelos
Idea principal

Distintos algoritmos capturan distintos patrones en los datos.

Un modelo puede cometer errores que otro no comete y clasificar correctamente los errores del otro

Combinar modelos suele mejorar robustez y precisión.

Esta idea es la base de los ensembles modernos.

Bagging

Reducir varianza promediando outputs de modelos algo distintos

1. El modelo se entrena varias veces sobre subconjuntos de los datos (vía bootstrapping)

2. Se combinan sus predicciones (max voting, averaging…)

Es la técnica usada para pasar de DT a RF

Boosting

Reducir Sesgo

1. El modelo se entrena varias veces de forma secuencial sobre subconjuntos de datos (por ejemplo sobre los mal clasificados en la iteración anterior)

2. Se combinan sus predicciones (weighted averaging…)

Stacking

Genera un meta-modelo mediante la combinación de **distintos** modelos

1. Los modelos “base” generan predicciones

2. Se entrena un modelo a partir de esas predicciones (las y de los modelo base son las x del meta-modelo)

   *predicciones probabilísticas: en vez de y = 0, y = 0.31, donde y es la probabilidad de pertenecer a clase 1

Con este enfoque podemos explotar las fortalezas de cada modelo

Stacking
Arquitectura tipo

Nivel 1:

- LDA
- XGBoost
- SVM
- ANN, etc.

Nivel 2 (meta-modelo)

- modelo simple (ej. regresión logística, Ridge regression, Gradient Boosting suave).

Entrada del meta-modelo: predicciones de los modelos base.

Stacking
Overfitting y leakage

Stacking requiere utilizar la validación cruzada de forma correcta

Si entrenamos el meta-modelo sobre predicciones del training original:

- aparece overfitting severo.

*Las predicciones sobre el training set tienen ya cierto sesgo, si entrenamos el metamodelo sobre eso, se producirá overfitting

Por eso el metamodelo se debe entrenar sobre las predicciones de los modelos en los datos de validation:

1- entrenamos modelos en train y predecimos en validación

2- entrenamos metamodelos en esas predicciones

Train-Dev-Test para Stacking

Entrenamiento (Train) | Model selec. (Dev) | Evaluación (Test)



Ajuste de hiperparametros y selección del mejor modelo

Entrenamiento meta-model, con los ejemplos out-of-fold (OOF) del Cross-Validation (CV) -> Validation Set

Validation/Dev set, son para seleccionar modelo o hiperparámetro

La predicción de algo entrenado en train sobre train, tiene sesgo

Entrenamos el meta-modelo en la predicción (insesgada) sobre validación de los modelos (OOF de cada iteración del CV)

No vamos a poder ajustar mucho el meta-modelo ya que no tenemos más validación, así que esa parte la optimizamos menos o nada. (o usamos un nested-CV)

Elección de modelos para Stacking/Blending

En el cross-validation de Train:

1- Buscar los mejores hiperparametros para cada modelo (como hemos hecho hasta ahora)

2- Calcular cómo de parecidos son los errores que comete cada modelo

correlación entre outputs por ejemplo:

- si dos modelos tienen una predicción con mucha correlación, aporta poco mezclados
- Si dos modelos tienen performance alta y están poco correlados, es una situación muy prometedora para stacking/blending

Variación del Stacking: Blending
Blending

Una simplificación del Stacking:

Igual que el Stacking pero sin Cross-Validation. Únicamente una división de datos

Usarlo cuando se quiera una implementación rápida, sino, usar Stacking

Ejemplo de Stacking/Blending

Netflix Prize: Competición de tres años, 1 millones de dólares al ganador

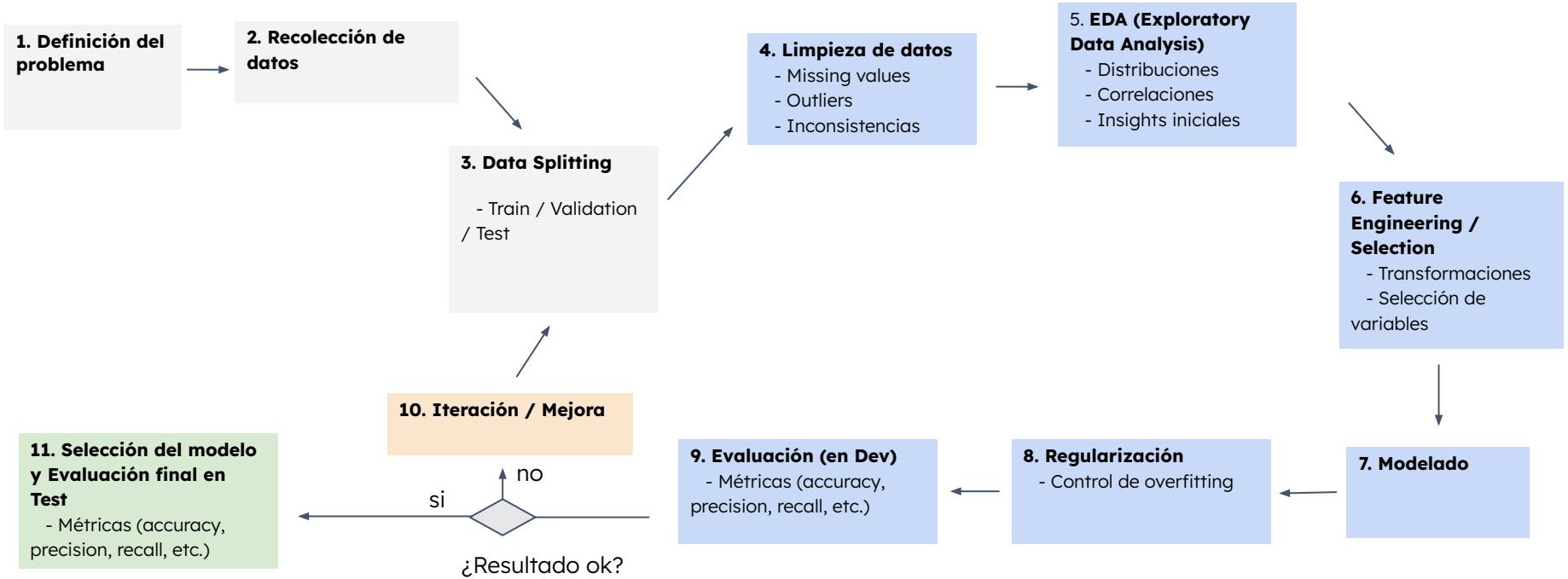
El ganador (BellKor’s Pragmatic Chaos), superó en un 10% al método anterior

La solución final era un modelo de Blending/Stacking sobre varios modelos (kNN, Matrix Factorization…)

Dudas?


NETFLIX
2009
DATE 09.21.09
PAY TO THE ORDER OF BellKor's Pragmatic Chaos
$ 1,000,000 00
AMOUNT ONE MILLION
00/100
FOR The Netflix Prize
Reed Hastings

Proyecto de ML
Pipeline clásico de un proyecto



MLOps

Carlos Bibián Nogueras, PhD
cbibian@itba.edu.ar |

Instituto Tecnológico de Buenos Aires

MLOps
Definición

ML Operations - nueva disciplina que incluye **herramientas y conceptos que ayudan durante el ciclo de vida de un proyecto de ML**.

Especialmente en deployment/modeling/data steps

Establece formas sistemáticos para plantear data/modeling y deployment

Motivación
Datos

Una skill muy importante en entrevistas del campo : ¿has desplegado alguna vez un algoritmo de ML en producción?

Objetivo: no sólo entrenar el algoritmo, sino también:

- ponerlo en producción,
- monitorearlo
- mantenerlo funcionando

En resumen, **haber hecho un deploy nos permite entender todo el ciclo de vida de un proyecto de ML**

¿Que rodea a nuestro algoritmo de ML?
Paisaje completo de un proyecto ML



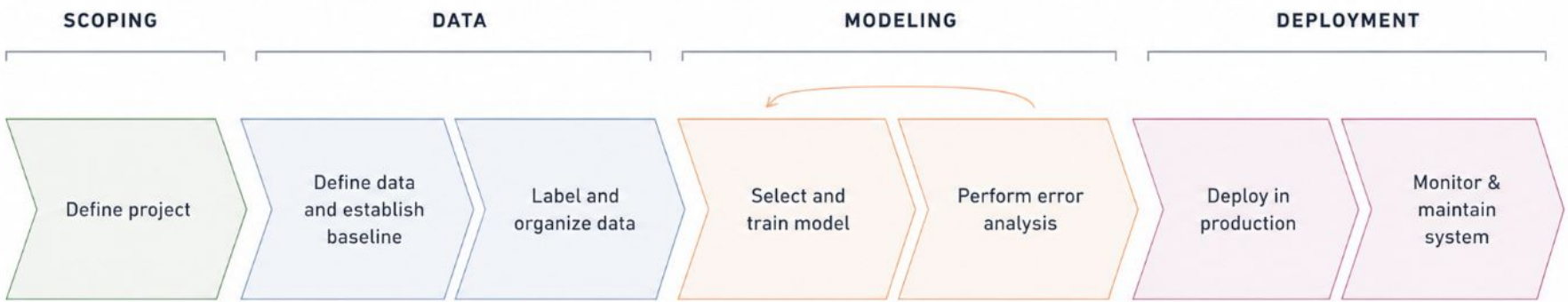
El **código de ML es un porcentaje bajo del código** de toda la aplicación final

Ese proof of concept que tenemos en el notebook, todavía necesita mucho codigo alrededor hasta ser implementado/desplegado en producción

POC to Production Gap

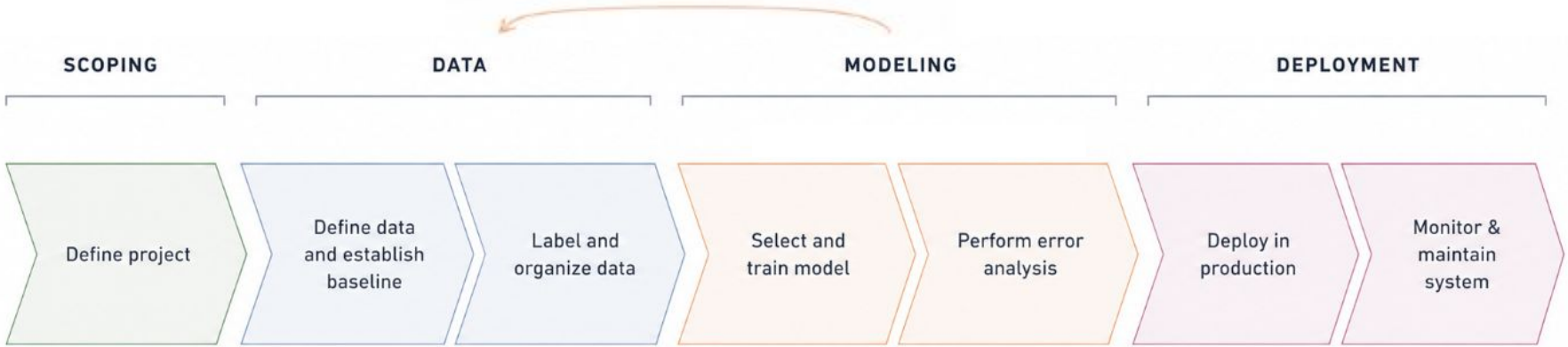
Ciclo de vida de un proyecto ML
The ML project lifecycle

Model-centered design

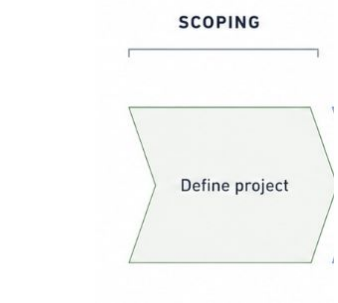


Ciclo de vida de un proyecto ML
The ML project lifecycle

Data-centered design



Paso a paso de un proyecto de ML
Datos

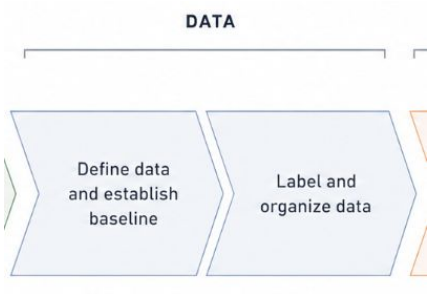


Scoping:

-Definir el proyecto, objetivos, alcance, etapas,

-Prioridades, errores que no cometer,

-Cómo interactuará con otros sistemas, que datos verá el sistema final…



Data:

- Datos y etiquetas consistentes,

- En dataset online, muchas veces los datos están ya listos para ser usados. En la realidad, hay un trabajo importante para mejorar la calidad de los datos (o al menos chequear),

Ejemplo: audios de speech recognition, vamos a normalizar volumen de audios? o brillo de imágenes etc.

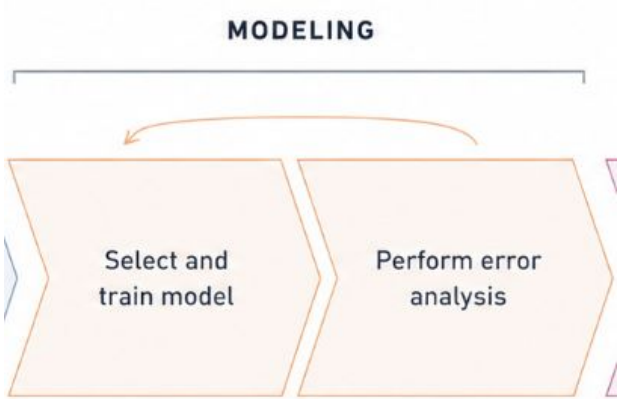
Paso a paso de un proyecto de ML

Modeling

Modelo+hiperparámetros+Datos-> sistema ML

- En academia/research: se suele centrar en modelos y ajuste de hiperparametros y no tocar los datos (muchas veces utiliza datasets predefinidos y estandarizados) - Model-Centric

-En producción/Deploy: suele ser más efectivo dejar el modelo fijo y centrarse en optimizar los datos y los hiperparametros - Data-Centric



Análisis de Errores

¿donde falla mi modelo?

-Esto me puede decir como puedo mejorar los datos, lo que nos puede dar un resultado de mejora grande del modelo.

-También informarte de qué tipo de datos necesitas si vas a grabar más, no solo grabar igual que lo que tienes, sino orientarlo mejor.

Paso a paso de un proyecto de ML

Deploy

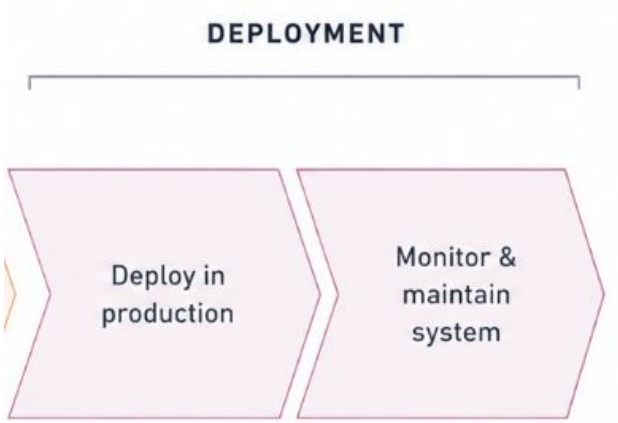
Implementar el modelo para que funcione en tiempo real, software, transformación de los datos…

Comprobar que los datos son comparables a aquellos con los que creamos el sistema -> Concept drift y Data drift

Monitoreo y Mantenimiento

¿Funciona mi modelo igual que durante el diseño y el deploy?

-Evaluar periódicamente varias métricas y aspectos para ver si el modelo funciona bien y poder detectar errores inesperados en el sistema



Concept and Data drift
Data Drift

Data drift: cambio en los datos que hace que el sistema esté viendo datos distintos a los que esperábamos

(Cambia la x)

Ejemplo:

- Cambios de iluminación en fotos de usuarios

- Speech recognition: quizás cambia el lenguaje un poco (lento), o calidad de los micrófonos de los smartphones de los usuarios

Concept and Data drift
Concept Drift

Concept drift: cambia el comportamiento que queremos modelar con el sistema

(Cambia la relación x-y)

Ejemplo:

- Covid 19: gente que no compraba online nunca, empezó a comprar mucho, y el muchos sistemas antifraude empezaron a detectar muchos fraudes (erróneamente).

-> Los sistemas tuvieron que ser reentrenados para funcionar en este nuevo ambiente

Tenemos que ser capaces de **controlar ambos durante y después del deploy**

Despliegue
Deployment

Carlos Bibián Nogueras, PhD
cbibian@itba.edu.ar |

Instituto Tecnológico de Buenos Aires

First deployment vs maintenance
Datos

Deploying son 2 tareas:

1) Software para el deploy

2) Mantenimiento y monitoreo (para comprobar data y concept drifts)

Deployment patterns
Algunos casos típicos

1. **Producto/servicio nuevo**: en este caso, lo ideal es empezar con poco tráfico

2. **Automatizar/asistir** en una tarea manual: veremos que aqui shadow mode deployment va perfecto

3. **Reemplazar un sistema de ML anterior**

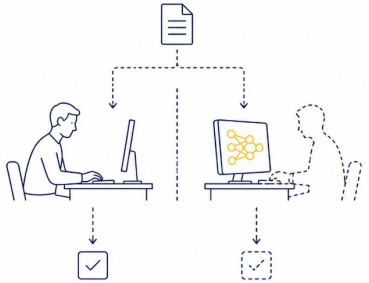
Ideas claves que comparten:

Crecimiento gradual de tráfico con monitoreo

Rollback: si vemos que no funciona como esperamos, podemos volver al modelo anterior

Tipos de Deployment
Shadow mode deployment

Shadow mode deployment



El sistema ML corre en paralelo con el trabajo del humano (shadow)

La **salida del sistema ML no se usa** para ninguna decisión en esta **etapa temprana**

Podemos **verificar** si estaría funcionando bien **y decidir si implementamos** el algoritmo para tomar decisiones de forma independiente o no

Es ideal para **tareas donde queremos asistir/automatizar una tarea humana**

Ejemplo: detección de piezas defectuosas

Tipos de deployment
Canary deployment



Canary deployment:

1) Usar solo una **fracción pequeña del tráfico** (por ejemplo 5%)

2) **Monitorea y ve incrementando gradualmente**. De esta forma, si hubiese algún error minimizamos el impacto en el proceso, y las pérdidas.

Cuando estamos listos para que el algoritmo tome **decisiones** de forma **independiente**, el canary deployment es el approach correcto

El nombre del método se debe a que los mineros llevaban consigo un canario para detectar fugas de gas (moría primero) y así saber cuándo tenían que salir rápido. el canario sería ese 5%...

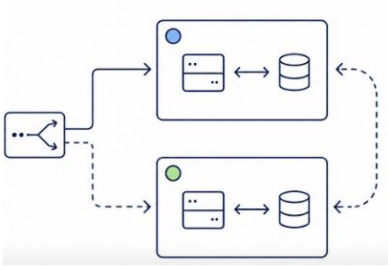
Tipos de deployment
Blue green deployment

Blue green deployment

Las **inputs** se mandan **a la nueva o la vieja versión de forma aleatoria.**

blue hace referencia a la versión anterior del sistema ML

green a la nueva versión



Ventajas:

- Permite **evaluar fácilmente la mejora de la nueva versión**
- Hacer **fácilmente** un **rollback a la version vieja** (blue), si la nueva no funcionase bien
- Podemos **aumentar** desde aquí **gradualmente** cuanto se envia al green version **si va bien**

Deployment patterns
Grados de automatización

Como vemos el **deployment no es un proceso 0 a 1, sino que es gradual**



Esto no quiere decir que haya que llegar a full automation siempre, se puede dejar el sistema en cualquier punto de Human in the loop (AI assistance o Partial automation).

Deployment
De Entrenamiento a Deploy

Hasta ahora hemos entrenado nuestros modelos y los hemos evaluado en nuestros notebooks.

Ahora vamos a simular un escenario más cercano a producción -> generando un código que vaya a ser corrido externamente

```
def predictor(X):
    ...
    return y_pred
```

La **función debe**:

- .Recibir los datos
- .Aplicar el preprocessing (p.ej escalado con mean y std de train)
- .Contener el modelo entrenado
- .Producir predicciones
- .Funcionar de forma independiente (sin acceso al notebook)
- .No usar Labels, ni reentrenar el modelo dentro ni utilizar variables globales del notebook...

Deployment
De Entrenamiento a Deploy

En ML real, el **entrenamiento** y la **inferencia** son dos etapas muy diferenciadas

El entrenamiento ocurre una vez*

La **inferencia** ocurre de forma **continua** y sobre **datos nuevos** en un entorno externo

El modelo solo es útil si **generaliza correctamente** y puede **ejecutarse externamente** e **integrarse en un pipeline real**.

Lo importante ocurre fuera del notebook

Monitoreo y Mantenimiento
Monitoring and Maintenance

Carlos Bibián Nogueras, PhD
cbibian@itba.edu.ar |

Instituto Tecnológico de Buenos Aires

Monitoring
Dashboards y métricas

Primer paso: Brainstorm de métricas importantes que necesitas monitorear para ver cómo funciona el sistema

Una forma de encontrar/definir estas métricas sería:

1. **Qué** cosas **pueden salir mal**?
2. **Qué** estadísticas/**métricas** me ayudarían a **detectar** ese problema

Las **métricas son bastante específicas del problema a resolver** así que es un proceso al que debemos dedicar tiempo

No hay problema en **empezar con una cantidad elevada de métricas** y gradualmente ir quitando las que no nos resulten útiles

Monitoring
Ejemplos de métricas

Métricas de **Software**:

Memoria, Latencia, Server load…

Métricas de **ML performance**:

Input metrics: Avg input length/volumen (speech recognition)

x Avg image brightness

Número de missing values

Output metrics: #veces que modelo devuelve null

y #veces que user repite la búsqueda (no está satisfecho)

#veces que el usuario pasa de usar el speech recognition y usa typing para la misma query

Monitoring
Tholds para alarmas

Otra herramienta que tenemos es utilizar **umbrales y alarmas o flags**:

Podemos **reducir** alguna métrica únicamente **a un umbral que nos avise si se han cruzado**, pero esa métrica no mirarla tanto y **centrarnos en otras métricas más informativas para el diagnóstico**

Umbral tanto superior como inferior como ambos!

Esos umbrales **podrían tener que ser adaptados con el tiempo**!

p ej: software load is too high o accuracy of ML is too low

Mantenimiento del Modelo

Manual: un equipo de ML engineers reentrenan el modelo para solucionar los problemas que estaban produciendo ese problema

-> generalmente este es el método más utilizado

Automático: a partir de error análisis, automatizar un reentrenamiento del modelo que ataque las debilidades del sistema y los data drifts (obviamente con flags que nos avisen en casos más extremos/importantes)

Monitoring pipeline
Ejemplo de dos sistemas en serie

A veces tenemos dos sistemas de ML trabajando en serie (un detector de voz primero y luego un reconocimiento de speech).

Los cambios en el primero producen cambios en lo que le llega al segundo

user data (clickstream) -> user profile -> recommender systems -> recommendations

Si cambia el user data (x), hay un **efecto cascada** en user profile y por tanto en el recommender system.

Aquí las **métricas** bien elegidas son **aún más importantes** para **detectar qué** está pasando y **dónde**

Como hemos visto antes: software metrics, input metrics, output metrics y las intermedias entre sistemas

Métricas a través del tiempo

¿Cómo de rápido cambian los valores de las métricas?

Dos situaciones:

Los datos de usuario suelen **cambiar lento**, los patrones de compra o de preferencias no suelen cambiar de un dia para otro (excepciones: covid)

- suma agregada de decisiones individuales no suele cambiar bruscamente

En algunos aspectos de los negocios, los datos pueden **cambiar** más **rápido**, porque las empresas toman decisiones sobre hacer un teléfono u otro, un software u otro, y esto sí que pueden cambiar bruscamente

Reflexión final

Mensaje principal: **Construir un modelo no es lo mismo que crear un producto**

Los sistemas de IA reales tienen éxito cuando modelos, datos, infraestructura, deployment, monitoreo y trabajo humano se diseñan en conjunto

ML no es solo un **conjunto de modelos**, sino **también todo lo que los rodea**!